Assignment 5

Build a Support Ticket Classifier using Pre-trained Language Models from Hugging Face

1. Introduction

This project addresses the task of classifying customer support tickets using pre-trained language models from Hugging Face. The dataset consists of support tickets containing textual descriptions such as login issues, payment failures, delivery delays, and product-related problems.

The objective is to build an end-to-end transformer-based text classification pipeline that can analyse ticket content and assign it to the appropriate category.

The tickets are classified into the following labels:

- Incident
- Problem
- Request
- Change

The model takes ticket text as input and predicts the corresponding category based on its content.

2. Objective

The objective of this project is to:

- Build an end-to-end NLP pipeline for ticket classification
- Use a pre-trained transformer model (DistilBERT)
- Fine-tune the model on a real-world dataset
- Evaluate model performance using standard metrics
- Perform inference on new unseen inputs

3. Tools and Technologies

- **Python**
- **Pandas** – Data manipulation
- **Hugging Face Transformers** – Model and tokenizer
- **Hugging Face Datasets** – Dataset loading and processing
- **PyTorch** – Model training backend
- **Scikit-learn** – Evaluation metrics

4. Dataset

Dataset used: **Customer Support Tickets Dataset (Hugging Face)**

Each record contains:

- subject → Short title of ticket
- body → Detailed description
- type → Category label (target variable)

5. Data Preprocessing

5.1 Handling Missing Values

- Rows with missing labels (type) were removed
- Missing text fields were replaced with empty strings

5.2 Feature Engineering

The input text was created by combining:

```
Subject + Body
```

Example:

```
"Subject: Login issue Body: Unable to login to my account"
```

5.3 Label Encoding

Categorical labels were converted to numerical format:

Change → 0
Incident → 1
Problem → 2
Request → 3

5.4 Train-Test Split

The dataset was split into:

- 80% training
- 20% testing

5.5 Dataset Balancing

Initially, the model showed bias toward the "Incident" class.

To fix this:

- Equal number of samples were selected from each class
- This ensured balanced learning across categories

```
Balanced Data:
 type
Change      4007
Incident    4007
Problem     4007
Request     4007
Name: count, dtype: int64
```

6. Model Selection

Model used: distilbert-base-uncased

Why DistilBERT?

- Lightweight and faster than BERT
- Good performance on classification tasks
- Suitable for limited GPU

```
model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)
```

7. Tokenization

Text data was tokenized using Hugging Face tokenizer:

- Padding: max_length
- Truncation: Enabled
- Max length: 128 tokens

```
def tokenize_function(example):
    return tokenizer(
        example['text'],
        padding="max_length",
        truncation=True,
        max_length=128   # to speed up
    )

tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

```
Map:   0%|          | 0/16028 [00:00<?, ? examples/s]Map: 100%|          | 16028/16028 [00:02<00:
Map: 100%|          | 9718/9718 [00:01<00:00, 7221.56 examples/s]
```

Windows Update

Your device will restart soon (est. time to min)
Your device will restart at 19:42.
UST manages your update settings

8. Model Training

Training was performed using Hugging Face Trainer.

Training Configuration:

- Epochs: 3
- Batch size: 4

- Learning rate: 2e-5
- Optimizer: AdamW
- Device: GPU (CUDA)

```python
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="no",   #skip eval during training
    learning_rate=2e-5,
    per_device_train_batch_size=8,   # faster on GPU
    num_train_epochs=3,
    logging_dir="./logs",
    fp16=False
)
```

```
c:\Users\issac\Documents\Next GenAI assignment\venv\lib\site-packages\transformers\training_args.py:1474: FutureWarning: `evaluation_strategy` is deprecated and will be removed in version 4.46 of 🤗 Transformers. Use `eval_strategy` instead
  warnings.warn(
```

```python
trainer.train()
```

```
  8%|          | 500/6012 [04:30<1:07:09,  1.37it/s]{'loss': 0.6714, 'grad_norm': 2.2262983322143555, 'learning_rate': 1.8343313373253494e-05, 'epoch': 0.25}
 17%|          | 1000/6012 [11:27<1:10:08,  1.19it/s]{'loss': 0.4803, 'grad_norm': 2.180278778076172, 'learning_rate': 1.66833000665336e-05, 'epoch': 0.5}
 25%|          | 1500/6012 [20:01<1:30:33,  1.20s/it]{'loss': 0.4452, 'grad_norm': 4.4167938232421875, 'learning_rate': 1.5019960079840322e-05, 'epoch': 0.75}
 33%|          | 2000/6012 [28:39<1:00:20,  1.11it/s]{'loss': 0.4122, 'grad_norm': 10.821521759033203, 'learning_rate': 1.335662009314704e-05, 'epoch': 1.0}
 42%|          | 2500/6012 [36:24<54:23,  1.08it/s]  {'loss': 0.3613, 'grad_norm': 6.403250217437744, 'learning_rate': 1.169328010645376e-05, 'epoch': 1.25}
 50%|          | 3000/6012 [44:24<43:04,  1.17it/s]  {'loss': 0.3616, 'grad_norm': 0.15409740805625916, 'learning_rate': 1.0033266799733866e-05, 'epoch': 1.5}
 58%|          | 3500/6012 [52:32<46:53,  1.12s/it]  {'loss': 0.3526, 'grad_norm': 5.832420349121094, 'learning_rate': 8.369926813040587e-06, 'epoch': 1.75}
 67%|          | 4000/6012 [1:01:16<31:38,  1.06it/s]{'loss': 0.3474, 'grad_norm': 2.5058505535125732, 'learning_rate':
 75%|          | 4500/6012 [1:09:17<23:14,  1.08it/s]{'loss': 0.2904, 'grad_norm': 6.082468032836914, 'learning_rate':
 83%|          | 5000/6012 [1:17:20<16:38,  1.01it/s]{'loss': 0.2713, 'grad_norm': 11.309050559997559, 'learning_rate':
 91%|          | 5500/6012 [1:26:28<08:56,  1.05s/it]{'loss': 0.2796, 'grad_norm': 11.7098970413208, 'learning_rate': 1
100%|          | 6000/6012 [1:35:27<00:15,  1.32s/it]{'loss': 0.2763, 'grad_norm': 2.0807557106018066, 'learning_rate':
100%|          | 6012/6012 [1:35:43<00:00,  1.05it/s]{'train_runtime': 5743.3422, 'train_samples_per_second': 8.372, 't
0.3790901943753738, 'epoch': 3.0}

TrainOutput(global_step=6012, training_loss=0.3790901943753738, metrics={'train_runtime': 5743.3422, 'train_samples_per
47, 'total_flos': 1592447395295232.0, 'train_loss': 0.3790901943753738, 'epoch': 3.0})
```

9. Evaluation Metrics

The model was evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score

```python
trainer.evaluate(tokenized_datasets["test"])
```

```
100%|          | 1215/1215 [03:41<00:00,  5.48it/s]
{'eval_loss': 0.5081830620765686,
 'eval_accuracy': 0.7961514714961926,
 'eval_f1': 0.8002701569170358,
 'eval_precision': 0.8147892387029203,
 'eval_recall': 0.7961514714961926,
 'eval_runtime': 221.8912,
 'eval_samples_per_second': 43.796,
 'eval_steps_per_second': 5.476,
 'epoch': 3.0}
```

10. Results

Accuracy ≈ 0.80
F1 Score ≈ 0.80

Precision ≈ 0.81
Recall ≈ 0.80
Loss ≈ 0.51

Interpretation:

- The model correctly classifies ~80% of tickets
- Balanced performance across classes
- Good generalization on unseen data

11. Inference (Prediction)

A custom prediction function was implemented:

- Takes subject + body
- Returns:
  - Predicted label
  - Confidence score
  - Probability distribution

```python
#Incident
print(predict("Login issue", "Cannot login"))
print(predict("Late delivery", "My package is delayed"))
print(predict("Damaged product", "The item arrived broken"))
```

```
{'prediction': 'Problem', 'confidence': 0.5, 'all_probs': {'Change': 0.001, 'Incident': 0.499, 'Problem': 0.5, 'Request': 0.0}}
{'prediction': 'Incident', 'confidence': 0.852, 'all_probs': {'Change': 0.001, 'Incident': 0.852, 'Problem': 0.146, 'Request': 0.0}}
{'prediction': 'Incident', 'confidence': 0.966, 'all_probs': {'Change': 0.001, 'Incident': 0.966, 'Problem': 0.032, 'Request': 0.001}}
```

```python
# Problem
print(predict("Slow system", "The system is consistently slow every day"))
print(predict("Recurring bug", "This issue keeps happening after every update"))
print(predict("Performance issue", "Application performance has degraded over time"))
```

```
{'prediction': 'Problem', 'confidence': 0.939, 'all_probs': {'Change': 0.001, 'Incident': 0.059, 'Problem': 0.939, 'Request': 0.001}}
{'prediction': 'Problem', 'confidence': 0.61, 'all_probs': {'Change': 0.001, 'Incident': 0.389, 'Problem': 0.61, 'Request': 0.0}}
{'prediction': 'Incident', 'confidence': 0.525, 'all_probs': {'Change': 0.001, 'Incident': 0.525, 'Problem': 0.474, 'Request': 0.0}}
```

```python
# Request
print(predict("Access request", "Please provide access to the reporting dashboard"))
print(predict("Data export","Can you provide an option to download reports in CSV format"))
print(predict("Password reset", "I need to reset my account password"))
```

```
{'prediction': 'Request', 'confidence': 0.521, 'all_probs': {'Change': 0.424, 'Incident': 0.023, 'Problem': 0.032, 'Request': 0.521}}
{'prediction': 'Request', 'confidence': 0.997, 'all_probs': {'Change': 0.001, 'Incident': 0.001, 'Problem': 0.001, 'Request': 0.997}}
{'prediction': 'Change', 'confidence': 0.991, 'all_probs': {'Change': 0.991, 'Incident': 0.004, 'Problem': 0.003, 'Request': 0.001}}
```

```python
# Change
print(predict("Update config", "Please change system configuration settings"))
print(predict("Upgrade system", "We need to upgrade to the latest version"))
print(predict("Modify workflow", "Request to modify approval workflow"))
```

```
{'prediction': 'Change', 'confidence': 0.999, 'all_probs': {'Change': 0.999, 'Incident': 0.0, 'Problem': 0.0, 'Request': 0.0}}
{'prediction': 'Change', 'confidence': 0.999, 'all_probs': {'Change': 0.999, 'Incident': 0.0, 'Problem': 0.0, 'Request': 0.0}}
{'prediction': 'Change', 'confidence': 0.999, 'all_probs': {'Change': 0.999, 'Incident': 0.0, 'Problem': 0.0, 'Request': 0.0}}
```

12. Observations

- The model performs very well on clearly defined categories
- Some overlap exists between:

    - Incident vs Problem
    - Request vs Change

This is expected due to semantic similarity in real-world data

13. Challenges Faced

- Missing values in dataset
- Class imbalance leading to biased predictions
- Slow training on limited GPU
- Model initially predicting only one class

14. Solutions Implemented

- Handled missing values properly
- Balanced dataset across all classes
- Reduced input size for faster training
- Increased training epochs for better learning